

Log Analysis, Storage & Security

[Commands](#) · [File Locations](#) · [Rotation](#) · [Security](#) · [Severity Levels & Use Cases](#) —
[A to Z](#)

A practical Linux/RHEL-focused reference covering how logs are generated, where they live, how to read and analyze them, how they're rotated and stored long-term, how to secure them, and how severity/priority levels map to real-world use cases.

Prepared as a reference guide · AI BlockMaker style

Contents

- 1. What Is Logging & Why It Matters
- 2. Log File Locations (Linux / RHEL / systemd)
- 3. Core Log Analysis Commands
- 4. journalctl Deep Dive (systemd journal)
- 5. Advanced Filtering & Parsing (grep, awk, sed, regex)
- 6. Log Storage, Rotation & Retention (logrotate, rsyslog)
- 7. Centralized / Remote Logging (rsyslog forwarding, ELK, Loki)
- 8. Log Security & Integrity (permissions, SELinux, auditd, AIDE)
- 9. Syslog Severity (Priority) Levels & Use Cases
- 10. Syslog Facilities Reference
- 11. Command Cheat-Sheet by Task Type
- 12. A-to-Z Glossary of Logging Terms & Commands

1. What Is Logging & Why It Matters

A log is a timestamped record of an event generated by the kernel, a system daemon, an application, or a security subsystem. Logging exists to answer three questions after the fact: **what happened**, **when it happened**, and **what caused it**. Good logging practice sits at the intersection of three disciplines:

- **Analysis** — reading and correlating logs to debug an incident, an outage, or a performance issue.
- **Storage** — deciding how long logs live, how they rotate, how they're compressed, and where they're archived.
- **Security** — making sure logs are tamper-evident, access-controlled, and useful as forensic evidence.

Why it matters in production environments:

- Root-cause analysis of crashes, failed services, and application errors.
- Security auditing — detecting brute-force attempts, privilege escalation, unauthorized file access.
- Compliance (PCI-DSS, HIPAA, ISO 27001, SOC 2) — most standards require log retention and tamper protection.
- Capacity planning — disk, memory, and network trend analysis over time.
- Building alerting pipelines (e.g., feeding into SIEM tools like Splunk, ELK, Wazuh, Graylog).

2. Log File Locations (Linux / RHEL / systemd)

Most classic logs live under **/var/log/**. On RHEL/CentOS/Fedora systems using systemd, a large portion of logging also flows through the binary **systemd-journald** journal rather than flat text files.

Path	What it contains
/var/log/messages	General system messages (RHEL/CentOS). Debian/Ubuntu equivalent: /var/log/syslog
/var/log/secure	Authentication & authorization events — sshd, sudo, su (RHEL). Debian: /var/log/auth.log
/var/log/audit/audit.log	SELinux & auditd kernel-level audit trail (very security-relevant)
/var/log/boot.log	Messages generated during system boot
/var/log/dmesg	Kernel ring buffer — hardware, driver, and boot-time kernel messages
/var/log/cron	Cron job execution logs (RHEL). Debian keeps this inside syslog
/var/log/maillog	Mail server activity (Postfix, Sendmail)
/var/log/httpd/ or /var/log/apache2/	Apache access_log and error_log
/var/log/nginx/	Nginx access.log and error.log
/var/log/mysqld.log or /var/log/mysql/	Database server logs (MySQL/MariaDB)
/var/log/yum.log or /var/log/dnf.log	Package installation/update history (RHEL/Fedora)

Path	What it contains
/var/log/firewalld	firewalld service logs
/var/log/wtmp, /var/log/btmp, /var/log/lastlog	Binary login records — read via last, lastb, lastlog commands
/run/log/journal/ or /var/log/journal/	Persistent systemd journal (binary), read via journalctl
~/.bash_history	Per-user shell command history (not a syslog, but often audited)
/var/log/containers/ /var/log/pods/	Kubernetes container/pod logs on a node

Tip: Whether a system uses flat-file logs under /var/log or the systemd journal (or both) depends on the rsyslog/journald configuration in /etc/rsyslog.conf and /etc/systemd/journald.conf. Many distros run both side-by-side, with rsyslog reading from the journal and writing human-readable copies to /var/log for compatibility.

3. Core Log Analysis Commands

These are the everyday commands used to view, search, and slice log files.

Command	Purpose
cat file.log	Print entire file to stdout — fine for small files, avoid on huge logs
tail -n 50 file.log	Show the last 50 lines
tail -f file.log	Follow a log live as new lines are appended (real-time monitoring)
tail -F file.log	Like -f but re-attaches if the file is rotated/recreated
head -n 50 file.log	Show the first 50 lines
less file.log	Page through a large file interactively (search with /pattern)
grep 'error' file.log	Print lines containing a pattern
grep -i 'error' file.log	Case-insensitive search
grep -c 'error' file.log	Count matching lines instead of printing them
grep -v 'debug' file.log	Invert match — show lines NOT containing pattern
grep -A5 -B2 'panic' file.log	Show 5 lines after and 2 lines before each match (context)
egrep '(error fail denied)' file.log	Search for multiple patterns using extended regex
zcat file.log.gz	Read a gzip-compressed rotated log without decompressing to disk
zgrep 'pattern' file.log.gz	grep directly inside a compressed log
sort file.log uniq -c sort -nr	Count and rank duplicate lines — great for finding top offenders
awk '{print \$1,\$5}' file.log	Extract specific fields/columns from structured log lines

Command	Purpose
<code>sed -n '100,200p' file.log</code>	Print only a specific line range
<code>wc -l file.log</code>	Count total lines (events) in a log file
<code>watch -n 2 'tail -20 file.log'</code>	Re-run a command every 2 seconds — poor-man's live dashboard
<code>logger 'test message'</code>	Manually inject a message into the system log (useful for testing pipelines)

4. journalctl Deep Dive (systemd journal)

On systemd-based distros (RHEL 8/9, Fedora, Ubuntu 16.04+), **journalctl** is the primary tool for querying the binary journal maintained by systemd-journald.

Command	Purpose
<code>journalctl</code>	Show the entire journal, oldest first
<code>journalctl -e</code>	Jump to the end (most recent entries) — like less at EOF
<code>journalctl -f</code>	Follow new entries live, similar to tail -f
<code>journalctl -u sshd</code>	Show logs only for the sshd unit/service
<code>journalctl -u nginx --since today</code>	Logs for a unit since the start of today
<code>journalctl --since '2026-07-10' --until '2026-07-12'</code>	Show logs within a date range
<code>journalctl -p err</code>	Show only entries at priority 'err' or higher severity
<code>journalctl -k</code>	Show only kernel messages (equivalent of dmesg)
<code>journalctl -b</code>	Show logs from the current boot only
<code>journalctl -b -1</code>	Show logs from the previous boot
<code>journalctl --list-boots</code>	List all boot IDs the journal has recorded
<code>journalctl _PID=1234</code>	Filter by a specific process ID
<code>journalctl -o json-pretty</code>	Output structured JSON (useful for scripting/SIEM ingestion)
<code>journalctl --disk-usage</code>	Show how much disk space the journal is consuming
<code>journalctl --vacuum-size=500M</code>	Shrink the journal down to a target size
<code>journalctl --vacuum-time=2weeks</code>	Delete journal entries older than 2 weeks

Persistence: by default the journal may be volatile (memory-only, cleared on reboot). To make it persistent, set **Storage=persistent** in `/etc/systemd/journald.conf` and ensure `/var/log/journal/` exists, then restart `systemd-journald`.

5. Advanced Filtering & Parsing (grep, awk, sed, regex)

Real log analysis is about extracting a signal from noise. Combine tools with pipes.

Find the top 10 IPs hitting an Nginx/Apache access log:

```
awk '{print $1}' access.log | sort | uniq -c | sort -nr | head -10
```

Find all failed SSH login attempts and the offending IPs:

```
grep 'Failed password' /var/log/secure | awk '{print $(NF-3)}' | sort | uniq -c | sort -nr
```

Extract only 4xx/5xx HTTP errors from a web server log using regex:

```
grep -E '" (4|5)[0-9]{2} ' access.log
```

Search across multiple rotated + compressed logs at once:

```
zgrep -h 'OutOfMemory' /var/log/app/*.log*
```

Use sed to strip timestamps for easier diffing between two log snapshots:

```
sed -E 's/^[A-Za-z]{3} +[0-9]+ [0-9:]+ //' file.log > stripped.log
```

6. Log Storage, Rotation & Retention

Logs grow forever unless something manages them. On most Linux systems that job belongs to **logrotate**, driven by cron/systemd timers, plus rsyslog/journald's own retention settings.

logrotate

Config lives in /etc/logrotate.conf and per-service drop-ins under /etc/logrotate.d/.

```
/var/log/myapp/*.log {
daily
rotate 14
compress
delaycompress
missingok
notifempty
create 0640 appuser adm
sharedscripts
postrotate
systemctl reload myapp >/dev/null 2>&1 || true
endscript
}
```

Directive	Meaning
daily / weekly / monthly	How often to rotate
rotate N	How many old rotated copies to keep
compress / delaycompress	gzip old logs; delay compressing the most recent rotation by one cycle
size 100M	Rotate once a file crosses a size threshold, regardless of schedule

Directive	Meaning
create MODE OWNER GROUP	Recreate the log file fresh with correct permissions after rotation
missingok	Don't error out if the log file is missing
notifempty	Skip rotation if the log file is empty
postrotate/endscript	Run a command after rotation (e.g., reload a daemon so it reopens the file handle)

rsyslog

rsyslog (/etc/rsyslog.conf and /etc/rsyslog.d/*.conf) decides which facility/priority combinations get written to which files — this is the classic mechanism behind /var/log/messages, /var/log/secure, /var/log/maillog, etc.

```
# /etc/rsyslog.d/10-myapp.conf
# facility.priority destination
auth,authpriv.* /var/log/secure
mail.* -/var/log/maillog
cron.* /var/log/cron
local0.* /var/log/myapp/custom.log
*.emerg :omusrmsg:*
```

Retention strategy checklist

- Decide retention windows per compliance need (e.g., 90 days hot, 1 year cold/archived).
- Compress rotated logs (gzip/zstd) to save 80-90% of disk space.
- Move cold logs to cheaper storage (object storage / archive tier) rather than deleting immediately.
- Monitor disk usage of /var/log with 'du -sh /var/log/*' and set alerts before disks fill up.
- For journald, cap size with SystemMaxUse= in journald.conf so the journal can't consume the whole disk.

7. Centralized / Remote Logging

On any system with more than a few servers, shipping logs to a central store beats SSH-ing into each box individually.

Tool / Method	Role
rsyslog forwarding	Point rsyslog at a remote collector: *.* @logserver.example.com:514 (TCP) or @ for UDP
syslog-ng	Alternative to rsyslog with more flexible routing/filtering DSL
ELK / Elastic Stack	Filebeat/Logstash ship logs → Elasticsearch stores/indexes → Kibana visualizes
Grafana Loki + Promtail	Lightweight, label-based log aggregation designed to pair with Grafana + Prometheus
Graylog	Open-source SIEM-style centralized log management with alerting

Tool / Method	Role
Wazuh	Security-focused log analysis/SIEM built on the Elastic stack
journal remote (systemd-journal-remote)	Native systemd way to forward/receive journal entries between machines

Why centralize: correlate events across multiple hosts, survive a single host's disk/compromise, and give a single pane of glass for both debugging and security monitoring.

8. Log Security & Integrity

Logs are often the first thing an attacker tries to erase or tamper with — protecting them is as important as collecting them.

File permissions & ownership

```
# Typical safe ownership/permissions
chown root:adm /var/log/secure
chmod 640 /var/log/secure
# Restrict who can read auth-sensitive logs
ls -l /var/log/secure /var/log/audit/audit.log
```

- Most /var/log files should be root-owned, group 'adm' or 'root', mode 640 (not world-readable).
- Never make application log directories world-writable — that allows log injection/tampering.
- Use dedicated service accounts so app logs aren't written as root unnecessarily.

Immutability & tamper-evidence

```
# Make an archived log file immutable – even root can't modify/delete it
# without first removing the attribute
chattr +i /var/log/archive/2026-06.log.gz
lsattr /var/log/archive/2026-06.log.gz
```

- chattr +i sets the immutable bit at the filesystem level (ext4/xfs with extended attrs).
- AIDE (Advanced Intrusion Detection Environment) can hash log files and alert on unexpected changes.
- Consider write-once storage (WORM) or shipping logs off-host immediately — an attacker who gets root locally still can't rewrite history on a remote, access-controlled log server.

auditd — kernel-level audit trail

auditd (/etc/audit/auditd.conf, rules in /etc/audit/rules.d/) is the RHEL/CIS-benchmark standard for security auditing — tracking file access, privilege use, and syscalls.

Command	Purpose
auditctl -w /etc/passwd -p wa -k passwd_changes	Watch a file for writes/attribute changes, tag with a key
ausearch -k passwd_changes	Search audit log for events tagged with a given key
ausearch -m USER_LOGIN	Search for a specific audit event type (e.g., login attempts)

Command	Purpose
aureport --auth	Summarized authentication report from the audit log
auditctl -l	List currently loaded audit rules
systemctl status auditd	Confirm the audit daemon itself is running

SELinux context

```
ls -Z /var/log/secure
restorecon -v /var/log/secure # fix a mislabeled log file's SELinux context
```

Security checklist

- Ship logs to a remote/central collector so a compromised host can't erase its own evidence.
- Sign or hash log batches (e.g., sha256sum) before archiving so tampering is detectable.
- Restrict who can run 'journalctl', read /var/log/audit, or clear logs (audit rule '-e 2' locks audit config).
- Alert on log-clearing commands themselves (e.g., 'journalctl --vacuum', 'rm /var/log/*', 'history -c').
- Encrypt logs at rest if they contain sensitive data (PII, credentials accidentally logged, tokens).
- Regularly review who has sudo access to log directories — treat log access itself as a privileged action.

9. Syslog Severity (Priority) Levels & Use Cases

Every syslog message carries a numeric **priority** combining a facility and a severity (0 = most severe, 7 = least severe). This is the standard defined by RFC 5424 / RFC 3164.

#	Severity	Meaning	Typical use case
0	Emergency (emerg)	System is unusable	Full system crash, kernel panic — page on-call immediately, top-tier alert
1	Alert (alert)	Action must be taken immediately	Database corruption detected, primary storage full — automated page/SMS
2	Critical (crit)	Critical conditions	Hardware failure, application component down — high-priority alert
3	Error (err)	Error conditions	Failed request, unhandled exception — logged and reviewed, may alert if frequent
4	Warning (warning)	Warning conditions	Deprecated API used, disk usage above 80% — dashboard/report, no page
5	Notice (notice)	Normal but significant condition	Service restarted, config reloaded — informational, worth noting
6	Informational (info)	Informational messages	Successful login, routine job completed — general audit trail
7	Debug (debug)	Debug-level messages	Verbose developer tracing — enabled only while actively troubleshooting

Practical rule of thumb: in production, run services at 'info' or 'notice' level day-to-day, temporarily raise verbosity to 'debug' only while actively diagnosing an issue (then revert it — debug-level logging can be extremely high-volume and expensive to store), and route 'crit'/'alert'/'emerg' straight into paging/alerting systems rather than relying on someone reading the file.

10. Syslog Facilities Reference

Facility identifies which subsystem generated the message; combined with severity it forms the full syslog priority.

Facility	Description
kern	Kernel messages
user	Generic user-level messages (default if a program doesn't specify one)
mail	Mail subsystem (Postfix, Sendmail)
daemon	System daemons without their own facility
auth / authpriv	Security/authentication messages (authpriv is often more restricted/private)
syslog	Messages generated internally by the syslog daemon itself
lpr	Printing subsystem
news	Usenet news subsystem (legacy)
cron	Cron/at job scheduler
local0–local7	Reserved for custom/site-specific use — commonly used by in-house applications

11. Command Cheat-Sheet by Task Type

Real-time monitoring

- `tail -f /var/log/messages`
- `journalctl -f -u nginx`
- `watch -n 5 'tail -20 /var/log/app.log'`

Historical search / forensics

- `journalctl --since '1 hour ago' -p err`
- `grep 'authentication failure' /var/log/secure`
- `zgrep -h 'OOM' /var/log/*.gz`
- `ausearch -ts today -m USER_LOGIN`

Statistics / trend analysis

- `awk '{print $1}' access.log | sort | uniq -c | sort -nr | head`

- `journalctl -p warning --since today | wc -l`
- `aureport --summary`

Storage & housekeeping

- `logrotate -f /etc/logrotate.conf` (force a rotation immediately)
- `journalctl --vacuum-time=30d`
- `du -sh /var/log/* | sort -rh | head`

Security auditing

- `auditctl -w /etc/shadow -p wa -k shadow_watch`
- `lsattr /var/log/secure`
- `aureport --auth --summary`

12. A-to-Z Glossary of Logging Terms & Commands

#	Term	Description
A	auditd	Linux kernel-level audit daemon for security event tracking.
A	AIDE	File/log integrity checker that alerts on unexpected changes via hashing.
B	btmpt	Binary log of failed login attempts, read with 'lastb'.
C	chattr +i	Marks a file immutable at the filesystem level, blocking edits/deletes.
C	Compression (gzip/zstd)	Shrinks rotated logs to save disk space, applied by logrotate.
D	dmesg	Displays the kernel ring buffer — hardware/driver/boot messages.
E	ELK Stack	Elasticsearch + Logstash + Kibana — search, ingest, and visualize logs centrally.
F	Facility	Category of the log source (auth, cron, mail, kern, local0-7, etc.).
F	Filebeat	Lightweight log shipper that forwards files to Logstash/Elasticsearch.
G	grep	Pattern-search utility; the most-used log analysis command.
H	HRFlowable-style rotation	Daily/weekly rotation policy set in logrotate config.
I	Immutable bit	Filesystem attribute preventing modification, even by root.
J	journalctl	Query tool for the systemd binary journal.
J	journald	systemd's logging daemon that collects/stores the journal.
K	Kernel ring buffer	In-memory circular buffer of kernel messages, viewed via dmesg.
L	lastlog / last / lastb	Commands showing recent successful/failed logins.
L	logrotate	Utility that rotates, compresses, and prunes log files on a schedule.
L	Loki	Grafana's log aggregation system using labels instead of full-text indexing.
M	Maillog	Log file for mail transfer agent activity (Postfix/Sendmail).
N	Notice	Syslog severity 5 — normal but noteworthy event.
O	omusrmsg	rsyslog output module that sends a message directly to a logged-in user's terminal.
P	Priority (PRI)	Combined facility + severity value carried in a syslog message header.
P	Persistent journal	journald storage mode that survives reboots (Storage=persistent).
Q	Query filters (journalctl)	Flags like -u, -p, --since/--until used to narrow journal output.
R	rsyslog	Modern syslog daemon that routes messages to files, remote hosts, or programs.
R	Retention policy	Rules defining how long logs are kept before deletion/archival.
S	SELinux context	Security label on log files enforced independently of Unix permissions.

#	Term	Description
S	Severity	Numeric urgency level of a log message (0 = emergency to 7 = debug).
S	syslog-ng	Alternative syslog daemon with an advanced filtering/routing DSL.
T	tail -f	Follows a growing log file in real time.
U	uniq -c	Counts consecutive duplicate lines — often combined with sort for ranking.
V	--vacuum-time / --vacuum-size	journalctl flags to prune the journal by age or size.
W	wtmp	Binary log of all logins/logouts and reboots, read via 'last'.
W	WORM storage	Write-Once-Read-Many storage used to make archived logs tamper-proof.
X	XFS/ext4 extended attributes	Filesystem features enabling chattr immutability on log files.
Y	YUM/DNF log	/var/log/yum.log or dnf.log — package install/update history on RHEL/Fedora.
Z	zgrep / zcat	Search or read compressed (.gz) rotated log files without manual decompression.

End of guide. Built for hands-on Linux/RHEL homelab use — pair this with journald + rsyslog configs you already run (e.g., BIND, Foreman/Katello, Ansible-managed nodes) to practice log analysis and security auditing in your own environment.